Entwicklung eines Entscheidungsmodells für Web-Architekturen: Eine vergleichende Analyse von Framework-Overhead und nativen Technologien hinsichtlich Energieeffizienz, Performance und Langlebigkeit

Bachelorarbeit
im Studiengang Medieninformatik
an der Fakultät für Informatik und Ingenieurwissenschaften
der Technischen Hochschule Köln

vorgelegt von:	Stefan Nikolic
Matrikel-Nr.:	11155490
Adresse:	Koblenzer Str. 24 57610 Almersbach stefan.nikolic@smail.th-koeln.de
Erstprüfer:	Prof. Christian Noss
Zweitprüfer:	Prof. Dr. Hoai Viet Nguyen

Almersbach, 19.02.2026

Kurzfassung/*Abstract*

Kurzfassung

Die moderne Web-Entwicklung ist durch eine hohe Komplexität und eine stetig wachsende Anzahl an Architekturoptionen geprägt. Entwickler stehen regelmäßig vor der Herausforderung, zwischen der maximalen Effizienz nativer Web-Technologien und dem strukturellen Komfort moderner Frameworks abzuwägen. Ziel dieser Bachelorarbeit ist die Entwicklung eines mathematisch fundierten Entscheidungsmodells, das diese Wahl objektiviert.

Methodisch basiert die Arbeit auf einer systematischen Literaturanalyse zur Identifikation technischer und organisatorischer Auswahlkriterien. Diese Daten bilden die Basis für ein hybrides Entscheidungsmodell, welches den *Analytic Hierarchy Process* (AHP) zur individuellen Kriteriengewichtung mit der *Technique for Order of Preference by Similarity to Ideal Solution* (TOPSIS) zur Alternativenbewertung kombiniert. Im Rahmen der Arbeit wurde ein webbasiertes Artefakt entwickelt, das diesen Prozess automatisiert und für die Praxis zugänglich macht.

Die Evaluierung anhand verschiedener Projektszenarien zeigt, dass das Modell erfolgreich zwischen technischen Idealen und organisatorischen Notwendigkeiten differenziert. Es wird deutlich, dass es keine universelle Bestlösung gibt, sondern die optimale Architektur stets vom spezifischen Anforderungsprofil abhängt. Das entwickelte Modell leistet somit einen Beitrag zur Professionalisierung der Architekturauswahl in der Softwareentwicklung.

Schlagwörter: Web-Entwicklung, Softwarearchitektur, JavaScript-Frameworks, Entscheidungsmodell, MCDA, AHP, TOPSIS.

Inhaltsverzeichnis

Tabellenverzeichnis **V**

Abbildungsverzeichnis **VI**

Abkürzungsverzeichnis **VII**

Glossar **VIII**

1. Einleitung **1**
1.1. Problemstellung und Relevanz 1
1.2. Zielsetzung der Arbeit 1
1.3. Forschungsfragen 2
1.4. Aufbau der Arbeit 3

2. Theoretische Grundlagen **4**
2.1. Web-Architekturen im Wandel 4
2.2. Native Web-Technologien 4
2.2.1. Beispiel: Native DOM-Manipulation 5
2.3. Framework-basierte Entwicklung 5
2.3.1. Beispiel: Abstraktion in React 6
2.4. Konzeptionelle Unterschiede 6
2.4.1. Direkter vs. abstrahierter API-Zugriff 6
2.4.2. Entwicklungsparadigmen: Imperativ vs. Deklarativ 7
2.5. Kriterien zur Bewertung von Web-Architekturen 7
2.5.1. Performance 7
2.5.2. Energieeffizienz 7
2.5.3. Wartbarkeit 8
2.5.4. Langlebigkeit 8
2.5.5. Team-Skalierbarkeit 8
2.5.6. Produktivität 9

3. Stand der Forschung und Analyse der Benchmarks **10**
3.1. Analyse technischer Metriken: Performance und Energieeffizienz 10
3.1.1. Laufzeit-Performance 10
3.1.2. Energieverbrauch und Ressourcenbedarf 11

3.2. Analyse software-ökonomischer und organisatorischer Faktoren 11
3.2.1. Wartbarkeit 12
3.2.2. Langlebigkeit 12
3.2.3. Team-Skalierbarkeit 12
3.2.4. Entwicklerproduktivität 12

4. Konzeption und Modellierung der Entscheidungsmatrix 14
4.1. Forschungsdesign und Vorgehensweise 14
4.2. Datentransformation und gewichtetes Scoring-Modell 15
4.2.1. Logik der Quellen-Gewichtung 15
4.2.2. Aggregation zu Endscores für die Entscheidungsmatrix 16
4.3. Analytic Hierarchy Process (AHP) zur Kriteriengewichtung 16
4.3.1. Hierarchische Struktur und paarweiser Vergleich 17
4.3.2. Mathematische Ableitung der Gewichte 17
4.3.3. Konsistenzprüfung 17
4.4. Das TOPSIS-Verfahren zur Alternativenbewertung 18
4.4.1. Prinzip der idealen und anti-idealen Lösung 18
4.4.2. Mathematischer Ablauf des Algorithmus 18

5. Implementierung des Artefakts 20
5.1. Systemarchitektur und Technologiestack 20
5.2. Workflow und Benutzeroberfläche 21
5.2.1. Landing Page 21
5.2.2. Phase 1: Subjektive Priorisierung mittels AHP 22
5.2.3. Visualisierung der Kriteriengewichte 23
5.2.4. TOPSIS-Ranking der Frameworks 23

6. Evaluierung und Diskussion 25
6.1. Szenariobasierte Evaluierung 25
6.1.1. Szenario 1: Enterprise Core-System (Online-Banking) 25
6.1.2. Szenario 2: Startup MVP (Minimum Viable Product) 26
6.1.3. Sensitivitätsanalyse und Modellrobustheit 28
6.2. Diskussion der Modellergebnisse 28
6.3. Limitationen des Modells 29

7. Fazit und Ausblick 30
7.1. Zusammenfassung der Ergebnisse 30
7.2. Kritische Würdigung 31
7.3. Ausblick 31

Literatur 32

A. Quellengewichtung und Datengrundlage **36**
A.1. Detaillierte Quellengewichtung 36
A.2. Einzelscores der Kriterien-Dimensionen 37

Tabellenverzeichnis

4.1. Finale Datenmatrix (Scores 1–10) für das TOPSIS-Verfahren 16

A.1. Datenerhebung: Quellen-Gewichtung 36
A.2. Datenerhebung: Team-Skalierbarkeit 37
A.3. Datenerhebung: Langlebigkeit 37
A.4. Datenerhebung: Performance 37
A.5. Datenerhebung: Energieeffizienz 38
A.6. Datenerhebung: Wartbarkeit 38
A.7. Datenerhebung: Produktivität 38

Abbildungsverzeichnis

5.1. Landing Page des Artefakts mit Erläuterung der Zielsetzung 21
5.2. Benutzeroberfläche für den paarweisen Kriterienvergleich (Analytic Hierarchy Process (AHP)) 22
5.3. Visualisierung der berechneten Kriteriengewichte 23
5.4. Ergebnis: Ranking der Frameworks 24
5.5. Vergleich vom besten Framework mit Technique for Order of Preference by Similarity to Ideal Solution (TOPSIS)-Ideal 24

6.1. Ergebnisauswertung: React als Sieger im Enterprise-Szenario 26
6.2. Ergebnisauswertung: Svelte als Sieger im MVP-Szenario 27

Abkürzungsverzeichnis

AHP Analytic Hierarchy Process.

API Application Programming Interface.

CPU Central Processing Unit.

CSS Cascading Style Sheets.

DOM Document Object Model.

HTML Hypertext Markup Language.

JS JavaScript.

JSON JavaScript Object Notation.

MCDA Multi-Criteria Decision Analysis.

MVP Minimum Viable Product.

NIS Negative Ideal Solution.

PIS Positive Ideal Solution.

TOPSIS Technique for Order of Preference by Similarity to Ideal Solution.

UI User Interface.

W3C World Wide Web Consortium.

Glossar

Breaking Changes Änderungen in einer neuen Software- oder Framework-Version, die nicht abwärtskompatibel sind und zwingende Anpassungen am bestehenden Projekt-Quellcode erfordern, damit die Anwendung weiterhin fehlerfrei funktioniert.

Bundle Size Die Gesamtgröße aller JavaScript-, CSS- und Asset-Dateien, die beim Aufruf einer Webanwendung über das Netzwerk an den Browser des Endnutzers übertragen werden müssen.

Diffing Ein algorithmischer Prozess, der den aktuellen Zustand der Benutzeroberfläche mit einem neuen Zielzustand vergleicht, um zu berechnen, welche minimalen Änderungen im realen DOM vorgenommen werden müssen.

Framework-Overhead Zusätzlicher Ressourcenverbrauch, der durch die Laufzeitumgebung und die Abstraktionsschichten eines Frameworks entsteht, im Vergleich zu einer nativen Implementierung.

Green IT Bestrebungen und Konzepte zur umwelt- und ressourcenschonenden Gestaltung der Informations- und Kommunikationstechnik, in der Web-Entwicklung primär fokussiert auf die Reduzierung der Leistungsaufnahme und des Datenverkehrs.

Prioritätsvektor Ein im AHP durch Matrix-Normalisierung berechneter Vektor (Eigenvektor), dessen einzelne Elemente die relative Wichtigkeit der untersuchten Kriterien abbilden und in der Summe exakt 1,0 ergeben.

Saaty-Skala Eine vom Mathematiker Thomas L. Saaty entwickelte numerische Skala (meist von 1 bis 9), die im AHP genutzt wird, um qualitative Präferenzen im paarweisen Vergleich in berechenbare, quantitative Werte zu übersetzen.

Time-to-Market Die Zeitspanne von der ersten Produktidee bis zur Markteinführung. In der Softwareentwicklung beschreibt dies die Geschwindigkeit, mit der ein funktionsfähiges Produkt ausgeliefert werden kann.

1. Einleitung

1.1. Problemstellung und Relevanz

Die moderne Web-Entwicklung ist geprägt von einer stetig steigenden Komplexität der Anforderungen an Interaktivität und Benutzererfahrung. JavaScript (JS) hat sich dabei als die fundamentale Programmiersprache des Webs etabliert und führt seit über einem Jahrzehnt die Rangliste der meistgenutzten Sprachen an (StackOverflow, 2025). Um dieser Komplexität gerecht zu werden, greifen Entwickler zunehmend auf Frameworks wie React, Angular, Vue.js und Svelte zurück, die durch komponentenbasierte Architekturen und Abstraktionsschichten wie den Virtual Document Object Model (DOM) die Entwicklung beschleunigen sollen (Ouchaib, 2024; Persson, 2020).

Trotz dieser Vorteile bringt die Nutzung von Frameworks technische Herausforderungen mit sich. Eines der zentralen Probleme ist der sogenannte Framework-Overhead. Dieser äußert sich in größeren Bundle-Größen und einem erhöhten Rechenaufwand auf der Client-Seite, was insbesondere bei weniger leistungsstarken Endgeräten zu Performance-Einbußen führen kann (Persson, 2020; Sinha & Jana, 2024). Studien verdeutlichen die wirtschaftliche Relevanz dieser Metriken: So konnte der Online-Händler Zalando nachweisen, dass eine Verbesserung der Ladezeit um lediglich 100 ms zu einer Steigerung des Kundenumsatzes um 0,7 % führte (Kučak et al., 2025).

Parallel dazu gewinnt die Energieeffizienz an Bedeutung. Da Webanwendungen einen erheblichen Teil des Informations- und Kommunikationstechnik Sektors ausmachen, rückt der CO2-Fußabdruck in den Fokus. Framework-basierte Anwendungen weisen oft einen deutlich höheren Energieverbrauch auf als native Technologien (Finckelsen, 2024). Zudem gefährdet die hohe Innovationsgeschwindigkeit im JS-Ökosystem die Langlebigkeit von Web-Architekturen, da Framework-Versionen schnell veralten (Diniz-Junior et al., 2022).

1.2. Zielsetzung der Arbeit

Das primäre Ziel dieser Bachelorarbeit ist die Entwicklung eines wissenschaftlich fundierten Entscheidungsmodells, das Web-Entwickler bei der Wahl zwischen nativen

Web-Technologien und Framework-basierten Architekturen unterstützt. Dabei soll nicht eine pauschale Empfehlung ausgesprochen, sondern eine differenzierte Analyse erfolgen, die über rein technische Aspekte hinausgeht. Das Modell berücksichtigt ein multidimensionales Kriteriensystem, bestehend aus technologischen Metriken (Performance, Energieeffizienz), software-ökonomischen Faktoren (Wartbarkeit, Langlebigkeit) sowie organisatorischen Anforderungen (Entwicklerproduktivität, Team-Skalierbarkeit).

Um dieses Ziel zu erreichen, wird ein methodischer Hybrid-Ansatz verfolgt, der in Form einer Web-Applikation umgesetzt wurde. Das Entscheidungsmodell integriert zwei etablierte Methoden der Multi-Criteria Decision Analysis (MCDA):

- **AHP:** Diese Methode wird eingesetzt, um die subjektiven Prioritäten des Entscheidungsträgers zu erfassen. Durch einen paarweisen Vergleich der Hauptkriterien (z. B. höhere Gewichtung von Performance gegenüber Langlebigkeit) wird eine individuelle Gewichtungsmatrix erstellt.
- **TOPSIS:** Auf Basis der durch den AHP ermittelten Gewichte bewertet die TOPSIS die verschiedenen technologischen Alternativen. Hierbei werden reale Benchmark-Daten aus der Literatur und empirischen Untersuchungen herangezogen, um die Distanz zur theoretisch idealen Lösung zu berechnen.

Das Ergebnis der Arbeit ist eine Entscheidungshilfe, die sowohl technische Metriken als auch projektspezifische Anforderungen mathematisch miteinander verknüpft. Damit wird ein Beitrag geleistet, die oft rein subjektiv getroffene Architekturauswahl in der Web-Entwicklung durch ein transparentes und reproduzierbares Modell zu ersetzen.

1.3. Forschungsfragen

Aus der oben dargelegten Problemstellung und der Zielsetzung dieser Arbeit ergibt sich eine zentrale Forschungsfrage, die den Kern der Untersuchung bildet:

> *Wie kann auf Basis bestehender wissenschaftlicher und praxisnaher Literatur ein Entscheidungsmodell entwickelt werden, das Entwickler bei der Wahl zwischen nativen Web-Technologien und Framework-basierten Web-Architekturen unter Berücksichtigung von technologischen, software-ökonomischen und organisatorischen Kriterien unterstützt?*

Zur Beantwortung dieser übergeordneten Fragestellung werden vier untergeordnete Forschungsfragen (F1–F4) definiert, die den strukturellen Aufbau der Arbeit leiten:

F1 – **Kriterienidentifikation:** Welche technischen und organisatorischen Kriterien werden in der Literatur als entscheidend für die Wahl einer Web-Architektur genannt?

F2 – **Einfluss von Frameworks:** Welche Vor- und Nachteile von Framework-basierten Web-Architekturen werden in Bezug auf Performance, Energieeffizienz und Wartbarkeit beschrieben?

F3 – **Langlebigkeit & Abhängigkeiten:** Wie beeinflussen Abhängigkeiten von Frameworks und nativen Technologien die langfristige Stabilität und Wartbarkeit von Webanwendungen?

F4 – **Modellbildung:** Wie lassen sich die identifizierten Kriterien in einer gewichteten Entscheidungsmatrix strukturieren, um eine objektive Architekturauswahl zu ermöglichen?

1.4. Aufbau der Arbeit

Die vorliegende Arbeit folgt einem strukturierten Ansatz von der theoretischen Fundierung bis zur praktischen Anwendung. Nach der Definition der technologischen Grundlagen in Kapitel 2 erfolgt in Kapitel 3 eine systematische Literaturanalyse, welche die Datenbasis für die Modellbildung liefert. Kapitel 4 beschreibt die Methodik des hybriden Entscheidungsmodells, indem der AHP zur Kriteriengewichtung mit der TOPSIS zur Alternativenbewertung kombiniert wird. Die technische Implementierung des webbasierten Artefakts wird in Kapitel 5 dokumentiert. Abschließend wird das Modell in Kapitel 6 anhand praxisnaher Szenarien evaluiert, bevor die Arbeit in Kapitel 7 mit einem Fazit und Ausblick schließt.

2. Theoretische Grundlagen

In diesem Kapitel werden die theoretischen Fundamente der Web-Entwicklung sowie die für das Entscheidungsmodell relevanten technologischen Konzepte erläutert. Dabei wird zunächst die historische Entwicklung von statischen Dokumenten hin zu komplexen Applikationsarchitekturen betrachtet, bevor eine detaillierte Definition der untersuchten Technologien und Metriken erfolgt.

2.1. Web-Architekturen im Wandel

Die Architektur von Webanwendungen hat sich von statischen Dokumenten zu komplexen Applikationsstrukturen entwickelt. In der Frühphase lag der Fokus auf der Präsentation von Informationen mittels Hypertext Markup Language (HTML) und Cascading Style Sheets (CSS), wobei JS nur für einfache, imperative Interaktionen genutzt wurde (Kučak et al., 2025; Rathinam, 2022).

Mit steigenden Anforderungen an die Interaktivität etablierte sich das Konzept der *Single Page Application*, bei dem die Anwendung einmalig geladen wird und weitere Interaktionen asynchron ohne kompletten Seiten-Reload erfolgen (Emmanni, 2023; Levlin, 2020). Dieser Paradigmenwechsel verlagerte die Logik zunehmend auf den Client. Um die resultierende Komplexität im Zustandsmanagement zu beherrschen, entstanden moderne Frameworks, die deklarative Modelle und Abstraktionen wie den Virtual DOM einführten (Ouchaib, 2024; Sinha & Jana, 2024). Dies steigerte zwar die Produktivität, erhöhte jedoch auch den technischen Overhead und den Energiebedarf der Anwendungen (Kučak et al., 2025; Wadholm, 2023).

2.2. Native Web-Technologien

Native Web-Technologien, oft als *Vanilla JS* bezeichnet, beschreibt die Entwicklung von Webanwendungen unter ausschließlicher Verwendung der vom Browser bereitgestellten Standards (HTML, CSS und JS). Im Kern steht hierbei der Verzicht auf externe Bibliotheken oder Frameworks, die über eine eigene Laufzeitumgebung verfügen (Persson, 2020).

Die zentrale Komponente ist das DOM. Gemäß der Definition des World Wide Web Consortium (W3C) ist das DOM eine plattform- und sprachneutrale Schnittstelle, die es Programmen ermöglicht, dynamisch auf den Inhalt, die Struktur und den Stil von Dokumenten zuzugreifen und diese zu aktualisieren (W3C, 1998). In diesem Modell wird die Webseite als Baumstruktur abgebildet, in der jedes Element ein manipulierbares Objekt darstellt (Mozilla Developer Network, 2025).

2.2.1. Beispiel: Native DOM-Manipulation

Ein typisches Beispiel für native Entwicklung ist die manuelle Aktualisierung der Benutzeroberfläche. Dieser Prozess erfordert die präzise, imperative Definition jedes einzelnen technischen Schrittes durch den Entwickler. Um beispielsweise ein neues Element zu einer bestehenden Liste hinzuzufügen, müssen die entsprechenden DOM-Knoten explizit selektiert und manipuliert werden:

Listing 2.1: Imperative DOM-Manipulation mit Vanilla JS

```
// 1. Elternelement im DOM suchen
const list = document.getElementById('myList');

// 2. Neues Element erzeugen
const newItem = document.createElement('li');
newItem.textContent = 'Neues Element';

// 3. Element explizit in den DOM-Baum einhaengen
list.appendChild(newItem);
```

2.3. Framework-basierte Entwicklung

Im Gegensatz zum nativen Ansatz bietet die framework-basierte Entwicklung eine vorgegebene Architektur und ein Set an Werkzeugen, welche die Erstellung komplexer Benutzeroberflächen abstrahieren. Frameworks übernehmen dabei nicht nur Hilfsfunktionen, sondern steuern maßgeblich den Kontrollfluss der Anwendung (Rathinam, 2022).

Das primäre Ziel dieser Technologien ist die Steigerung der Entwicklerproduktivität und die Beherrschung komplexer Anwendungszustände (*State Management*). Frameworks bieten hierfür oft ein komponentenbasiertes Modell an, bei dem das User Interface (UI) in kleine, wiederverwendbare Einheiten unterteilt wird, die ihren eigenen Zustand verwalten (Emmanni, 2023; Ouchaib, 2024).

2.3.1. Beispiel: Abstraktion in React

Frameworks wie React nutzen Konzepte wie den *Virtual DOM*, um die Interaktion mit dem Browser zu abstrahieren. Der Entwickler definiert das UI in einer beschreibenden Syntax (z. B. JSX). Bei einer Datenänderung berechnet das Framework intern die Differenz (*Diffing*) und aktualisiert den realen DOM automatisch (Xu, 2021). Das vorherige native Beispiel stellt sich in einem Framework deutlich deklarativer dar:

Listing 2.2: Deklarative UI-Erstellung in React (JSX)

```
function ItemList({ items }) {
  // Das Framework uebernimmt das Rendern und Aktualisieren der
      Liste
  return (
    <ul id="myList">
      {items.map(item => <li key={item.id}>{item.text}</li>)}
    </ul>
  );
}
```

2.4. Konzeptionelle Unterschiede

Die Wahl zwischen nativen und framework-basierten Ansätzen ist mit fundamentalen Unterschieden in der Art der Programmierung und dem Zugriff auf Systemressourcen verbunden.

2.4.1. Direkter vs. abstrahierter API-Zugriff

In einer nativen Architektur kommuniziert der Programmcode direkt mit der Engine des Browsers über standardisierte Web-Application Programming Interfaces (APIs) (z. B. Selectors API, Fetch API). Dies führt zu minimalen Verzögerungen, da keine zusätzliche Logik zwischengeschaltet ist (Persson, 2020). Frameworks hingegen führen eine Abstraktionsschicht ein. Der Zugriff auf die Browser-Engine erfolgt indirekt über die Framework-Laufzeitumgebung. Dies ermöglicht zwar eine höhere Developer Experience, erzeugt jedoch den sogenannten Framework-Overhead, da die Abstraktionslogik selbst Rechenleistung und Speicher beansprucht (Diniz-Junior et al., 2022; Wadholm, 2023).

2.4.2. Entwicklungsparadigmen: Imperativ vs. Deklarativ

Ein wesentlicher konzeptioneller Unterschied liegt im Programmierparadigma. Native Entwicklung ist primär **imperativ**: Der Entwickler beschreibt das „Wie“, also die exakte Abfolge der Befehle zur UI-Manipulation (Ouchaib, 2024). Frameworks hingegen folgen einem **deklarativen** Paradigma. Hier beschreibt der Entwickler das „Was“, also den gewünschten Endzustand des UI in Abhängigkeit von den Daten. Die Komplexität der Synchronisation zwischen Daten und Ansicht wird dabei an das Framework delegiert (Levlin, 2020; Ouchaib, 2024). Während der deklarative Ansatz die Wartbarkeit in großen Teams verbessert, verschleiert er oft die tatsächlichen Performance-Kosten der zugrunde liegenden Operationen.

2.5. Kriterien zur Bewertung von Web-Architekturen

Um die erste Forschungsfrage (F1) zu beantworten, werden im Folgenden jene technischen und organisatorischen Kriterien definiert, die in der Literatur als entscheidend für die Wahl einer Web-Architektur genannt werden. Diese sechs Dimensionen bilden die theoretische Grundlage für das spätere Entscheidungsmodell und stellen sicher, dass alle Aspekte der Softwarequalität und Projektökonomie abgedeckt sind.

2.5.1. Performance

Die Performance beschreibt die Effizienz der Anwendung und gilt als einer der primären Faktoren für die Akzeptanz einer Technologie (Pano et al., 2018). Dabei wird primär zwischen der technischen Ausführungsgeschwindigkeit von Operationen, wie der Laufzeit und der Dauer von DOM-Manipulationen, und dem Startup-Verhalten unterschieden. Letzteres misst die Zeit bis zur ersten Interaktionsmöglichkeit, welche maßgeblich durch die gewählte Architektur und initiale Ladezeiten beeinflusst wird (Kučak et al., 2025; Sinha & Jana, 2024).

2.5.2. Energieeffizienz

Die Energieeffizienz quantifiziert den ökologischen Fußabdruck und den physischen Ressourcenverbrauch auf dem Endgerät (Wadholm, 2023). Hierbei wird einerseits die direkte Leistungsaufnahme gemessen (meist in Milliwattstunden), um den Hardware-Energiebedarf bei der Code-Ausführung zu bestimmen (Finckelsen, 2024). Andererseits spielt die *Bundle Size* eine entscheidende Rolle: Die über das Netzwerk zu übertragende

Datenmenge korreliert direkt mit der Ladezeit und dem notwendigen Energieaufwand für den Datentransfer (Putra et al., 2025).

2.5.3. Wartbarkeit

Wartbarkeit definiert die Leichtigkeit, mit der Software verstanden, modifiziert und fehlerbereinigt werden kann. In empirischen Studien identifizieren Entwickler eine klare Code-Struktur und Modularität als kritisch für langfristige Projekte (Bauer, 2021; Duvander & Romhagen, 2019). Messbar wird dies durch strukturelle Metriken wie die *Lines of Code* oder die zyklomatische Komplexität zur Bestimmung der logischen Dichte (Mohammad et al., 2024). Darüber hinaus umfasst die Wartbarkeit das Software-Design im Allgemeinen, insbesondere den Grad der Isolation von Komponenten und die Qualität der verfügbaren Dokumentation (Zozas et al., 2019).

2.5.4. Langlebigkeit

Die Langlebigkeit bewertet die Zukunftssicherheit einer Technologie. Die Wahl einer reifen Architektur mit starkem Rückhalt minimiert das Risiko technischer Obsoleszenz und teurer Migrationen (Bauer, 2021; Pano et al., 2018). Ein wesentlicher Indikator hierfür ist die Ökosystem-Stabilität, welche sich durch hohe Community-Aktivität auf Plattformen wie GitHub und StackOverflow sowie durch die langfristige finanzielle und strukturelle Unterstützung durch große IT-Organisationen auszeichnet (Duvander & Romhagen, 2019; StackOverflow, 2025).

2.5.5. Team-Skalierbarkeit

Dieses Kriterium fokussiert sich auf die organisatorische Eignung einer Architektur. Die Verfügbarkeit von Fachkräften und die Eignung der Technologie für stetig wachsende Teams sind in der Industrie oft ausschlaggebender als technische Mikro-Optimierungen (Duvander & Romhagen, 2019; Pano et al., 2018). Bewertet werden hierbei Marktfaktoren wie die Verbreitung im Arbeitsmarkt, die Standardisierung von Architekturmustern, die Verfügbarkeit von Schulungsmaterial sowie die generelle Eignung für die kollaborative Entwicklung in Großprojekten (Bauer, 2021; Repolust, 2025).

2.5.6. Produktivität

Die Produktivität, in der Literatur auch als *Effort Expectancy* bezeichnet, beschreibt den Aufwand, der für die Erzielung eines funktionalen Ergebnisses nötig ist (Pano et al., 2018). Maßgebliche Treiber sind hierbei die Lernkurve – also die Zeitspanne, bis neue Entwickler die Technologie produktiv nutzen können (Bauer, 2021; Xu, 2021) – sowie das verfügbare Tooling. Die Qualität von Entwicklerwerkzeugen und modernen Debuggern hat einen massiven Einfluss darauf, wie schnell der Entwicklungs- und Testprozess durchlaufen werden kann (Duvander & Romhagen, 2019; Rathinam, 2022).

3. Stand der Forschung und Analyse der Benchmarks

In diesem Kapitel wird der aktuelle Forschungsstand bezüglich der Effizienz und Praktikabilität von Web-Architekturen analysiert. Ziel ist es, die in Kapitel 2 definierten Kriterien auf Basis existierender Studien quantitativ und qualitativ zu evaluieren. Diese Analyse liefert die empirische und numerische Datengrundlage für die in Kapitel 4 durchgeführte Modellbildung und das Scoring.

3.1. Analyse technischer Metriken: Performance und Energieeffizienz

Die nachfolgende Analyse der technischen Benchmarks dient der Beantwortung der zweiten Forschungsfrage (F2), indem die messbaren Auswirkungen der Framework-Abstraktion auf die Performance und Energieeffizienz untersucht werden.

3.1.1. Laufzeit-Performance

In der Literatur herrscht Einigkeit darüber, dass native Technologien in isolierten Ausführungsgeschwindigkeiten die absolute Basislinie bilden. Da Befehle direkt an die Browser-API übermittelt werden, weisen native Ansätze bei atomaren Operationen die geringsten Latenzen auf. So zeigt Persson (2020) in Vergleichsmessungen, dass Vanilla JS bei der Erstellung von 10000 Tabellenzeilen im Durchschnitt nur 221 Millisekunden benötigt, während Frameworks wie React durch den Overhead des DOM-Diffings für dieselbe Aufgabe oftmals mehr als die doppelte Rechenzeit 570 Millisekunden(React) bis 903 Millisekunden (Vue) beanspruchen.

Diese Größenordnungen werden durch groß angelegte Benchmarks wie den *js-framework-benchmark* bestätigt. Krause (2025) normiert hierbei die native Ausführung auf den Basiswert von 1,00. Frameworks wie React oder Angular weisen in diesen Messungen typischerweise Performance-Faktoren zwischen 1,30 und 1,50 auf (sind somit 30 % bis 50 % langsamer). Einen architektonisch abweichenden Ansatz verfolgt Svelte: Durch die Verlagerung der Abstraktion in den Kompilierungsprozess erzielt diese Technologie

Latenzen, die mit Werten zwischen 1,05 und 1,10 sehr nah an der nativen Baseline liegen und klassische Laufzeit-Frameworks signifikant übertreffen (Ouchaib, 2024; Putra et al., 2025).

3.1.2. Energieverbrauch und Ressourcenbedarf

Im Bereich der Green IT lässt sich der technische Overhead direkt in einen erhöhten Energiebedarf und einen größeren CO2-Fußabdruck übersetzen. Wadholm (2023) konnte durch vergleichende Messungen nachweisen, dass eine native Applikation (*Vanilla JS*) im direkten Vergleich zu einem React-Gegenstück deutlich ressourcenschonender agiert. Während die native Implementierung einen durchschnittlichen Energiebedarf von 12,83 Milliwattstunden (mWh) sowie geschätzte Emissionen von 5,5 Milligramm CO2-Äquivalent aufwies, verzeichnete die React-Anwendung bei identischem Funktionsumfang einen signifikant höheren Energiebedarf von 17,8 mWh und Emissionen von 7,97 Milligramm CO2-Äquivalent. Dies belegt quantitativ, dass die durch das Framework eingeführten Abstraktionsschichten in diesen Messreihen zu einem Mehrverbrauch von fast 39 % bei der reinen Leistungsaufnahme führen.

Zusätzlich verschlechtert sich die Energiebilanz durch die sogenannte Bundle Size. Während native Anwendungen primär nur den komprimierten, anwendungsspezifischen Code von wenigen Kilobyte (KB) übertragen, erfordern Frameworks den Download und das Parsen der jeweiligen Laufzeitbibliotheken. Wie Diniz-Junior et al. (2022) in vergleichenden Benchmarks nachweisen konnten, weisen Framework-basierte Applikationen signifikant größere Dateigrößen auf: Angular generiert hierbei das größte Datenpaket mit rund 196 KB, gefolgt von React mit ca. 135 KB und Vue mit knapp 127 KB. Diese zusätzliche Payload erzeugt einen direkten Anstieg des Energieaufwands beim Datentransfer über das Netzwerk sowie bei der anschließenden Bereitstellung und Ausführung im Arbeitsspeicher des Endgeräts.

3.2. Analyse software-ökonomischer und organisatorischer Faktoren

Dieser Abschnitt adressiert die dritte Forschungsfrage (F3). Während die Benchmarks die Überlegenheit nativer Ansätze verdeutlichen, zeigen organisatorische Studien, dass die Architekturentscheidung maßgeblich durch Marktdynamiken und Team-Strukturen beeinflusst wird (Duvander & Romhagen, 2019; Pano et al., 2018).

3.2.1. Wartbarkeit

Die Wartbarkeit korreliert stark mit dem Grad der technologischen Standardisierung. Best Practices werden durch Frameworks erzwungen, was laut Mohammad et al. (2024) die Fehlerdichte in großen Projekten signifikant senkt. Native Ansätze bieten maximale Flexibilität, überlassen dem Entwickler jedoch die volle Verantwortung für das Zustandsmanagement. Ohne strukturelle Leitplanken steigt in nativen Projekten bei zunehmender Komplexität die zyklomatische Code-Dichte drastisch an (vgl. Zozas et al., 2019). Entsprechend bewerten Entwickler in qualitativen Erhebungen von Bauer (2021) das in Frameworks integrierte Tooling für Tests und Architekturvalidierung als essenziell, um technische Schulden langfristig gering zu halten.

3.2.2. Langlebigkeit

Native Technologien weisen die höchste Langlebigkeit auf, da sie auf Web-Standards basieren, die vom W3C über Jahrzehnte strikt abwärtskompatibel gepflegt werden (Repolust, 2025). Framework-basierte Architekturen sind hingegen an Lebenszyklen und Major-Updates gebunden, welche oftmals *Breaking Changes* und damit kostenintensive Migrationsprojekte erzwingen (Repolust, 2025). Demgegenüber steht jedoch der Vertrauensfaktor: Pano et al. (2018) zeigen auf, dass die finanzielle Rückendeckung durch Tech-Giganten (wie Meta bei React oder Google bei Angular) von Entscheidern als ausreichende Garantie gegen einen plötzlichen Support-Wegfall gewertet wird.

3.2.3. Team-Skalierbarkeit

Ein zentraler Faktor für das Projektmanagement ist die Verfügbarkeit von Fachkräften. Hier offenbaren Frameworks ihren größten ökonomischen Nutzen, da sie als Industriestandards die Einarbeitungszeiten massiv verkürzen (Duvander & Romhagen, 2019). Diese Dominanz lässt sich quantifizieren: Gemäß der jährlichen Umfrage von StackOverflow (2025) geben über 40 % der professionellen Entwickler an, React aktiv zu nutzen. Spezialisierte, auf JS basierende native Eigenentwicklungen werden im Unternehmenskontext hingegen oft als Risikofaktor wahrgenommen, da projektspezifisches Wissen bei Personalwechseln schnell verloren geht (Bauer, 2021).

3.2.4. Entwicklerproduktivität

Die Produktivität wird durch das Zusammenspiel von Lernaufwand und Entwicklungsgeschwindigkeit bestimmt. Während die anfängliche Lernkurve (z. B. für Konzepte wie JSX in React oder TypeScript in Angular) steiler ausfällt als bei reinem JS, wird der

langfristige Aufwand (*Effort Expectancy*) bei der Skalierung von Features durch Frameworks drastisch gesenkt (Pano et al., 2018). Das umfangreiche Ökosystem an fertigen UI-Komponenten sowie mächtige für automatisierte Build-Prozesse überkompensieren die initialen Einarbeitungskosten und führen laut Emmanni (2023) in komplexen Projekten zu messbar kürzeren Release-Zyklen als rein native Umsetzungen.

4. Konzeption und Modellierung der Entscheidungsmatrix

Dieses Kapitel beschreibt die methodische Vorgehensweise bei der Entwicklung des Entscheidungsmodells. Ziel ist es, die in Kapitel 3 analysierten Erkenntnisse in ein mathematisch fundiertes System zu überführen, das eine objektive Architekturauswahl ermöglicht.

4.1. Forschungsdesign und Vorgehensweise

Zur Beantwortung der vierten Forschungsfrage (F4) wird ein hybrider Ansatz der MCDA gewählt. Die Auswahl einer Web-Architektur stellt ein komplexes Entscheidungsproblem dar, da verschiedene, teils konkurrierende Zielgrößen gleichzeitig berücksichtigt werden müssen (Riedl, 2005).

Um sowohl subjektive Projektanforderungen als auch objektive Daten in einem Modell zu vereinen, wird eine Kombination aus dem AHP und der TOPSIS verwendet. Die Wahl dieser spezifischen Kombination lässt sich durch die jeweiligen methodischen Stärken begründen:

- **Eignung des AHP für die Kriteriengewichtung:** Der AHP ist besonders geeignet, um komplexe Entscheidungsprobleme hierarchisch zu strukturieren und die oft subjektiven Präferenzen von Entscheidungsträgern mathematisch exakt abzubilden (Mühlbacher & Kaczynski, 2014). Durch den paarweisen Vergleich der Kriterien wird eine höhere Konsistenz in der Gewichtung erreicht, als dies durch eine rein intuitive Vergabe von Prozentwerten möglich wäre. Zudem bietet der AHP die Möglichkeit, die logische Stimmigkeit der Urteile über einen Konsistenzindex zu validieren (Riedl, 2005).
- **Stärken von TOPSIS bei der Alternativenbewertung:** Während der AHP bei einer hohen Anzahl von Alternativen sehr aufwendig wird, zeichnet sich TOPSIS durch eine hohe Recheneffizienz aus. Das Verfahren basiert auf dem Prinzip, dass die gewählte Alternative die kürzeste geometrische Distanz zur idealen Lösung und die größte Distanz zur negativen Ideallösung aufweisen

sollte (Madanchian & Taherdoost, 2023). Dies macht es ideal für den Vergleich technischer Frameworks auf Basis der in Kapitel 3 identifizierten Scores.

Das resultierende hybride Modell nutzt den AHP, um die individuelle Relevanz der sechs Dimensionen für ein spezifisches Projekt zu ermitteln, und wendet diese Gewichte anschließend im TOPSIS-Verfahren an, um die stabilen Benchmark-Daten gegen die Projektanforderungen zu prüfen. Dieser Ansatz minimiert die kognitive Überlastung des Entscheiders und führt zu einer mathematisch herleitbaren Empfehlung (vgl. Madanchian & Taherdoost, 2023; Mühlbacher & Kaczynski, 2014).

4.2. Datentransformation und gewichtetes Scoring-Modell

Ein zentraler Schritt der Modellierung ist die Überführung der qualitativen und quantitativen Erkenntnisse aus der Literaturanalyse in ein einheitliches numerisches System. Um die Reproduzierbarkeit der Ergebnisse zu gewährleisten, wurde ein gewichtetes Scoring-Verfahren entwickelt, das die Zuverlässigkeit der einzelnen Quellen systematisch berücksichtigt.

4.2.1. Logik der Quellen-Gewichtung

Die in Kapitel 3 analysierten Quellen fließen nicht mit gleicher Relevanz in die Berechnung der Endscores ein. Jede Quelle i wurde mit einem Vertrauensfaktor W_i versehen, der die Qualität und Aktualität der Daten abbildet. Diese Gewichtung basiert auf drei wissenschaftlichen Teilkriterien:

1. **Aktualität:** Berücksichtigung des schnellen Wandels im Web-Ökosystem.
2. **Methodischer Umfang (Scope):** Bewertung der Stichprobengröße oder des Benchmark-Umfangs.
3. **Wissenschaftlicher Grad:** Einstufung nach Publikationsart (z. B. Peer-Review vs. Survey).

Die vollständige Gewichtungstabelle aller verwendeten Quellen ist in Anhang dokumentiert (siehe A.1). W_i ergibt sich als arithmetisches Mittel der drei Teilkriterien, die jeweils auf einer Skala von 0,1 bis 1,0 bewertet wurden.

4.2.2. Aggregation zu Endscores für die Entscheidungsmatrix

Ein methodisches Problem beim Vergleich verschiedener Framework-Studien liegt in deren Heterogenität. Da die Versuchsaufbauten, Metriken (z. B. Millisekunden vs. Zyklen) und Testumgebungen zwischen den Quellen stark variieren, lassen sich Rohwerte verschiedener Studien nicht direkt miteinander vergleichen.

Um dieses Problem zu lösen, wurden die relativen Leistungen der Frameworks innerhalb jeder einzelnen Studie auf eine einheitliche Skala von 1 bis 10 transformiert. Diese Scores wurden anschließend unter Berücksichtigung der Quellengewichtung aggregiert. Die Berechnung des Endscores S für eine Kategorie erfolgt nach der Formel:

$$S = \frac{\sum_{i=1}^{n}(Score_i \cdot W_i)}{\sum_{i=1}^{n} W_i} \tag{4.1}$$

Dieses Verfahren ermöglicht es, mehrere Studien zu einem stabilen Gesamtwert zusammenzufassen, der die Basis für das spätere TOPSIS-Verfahren bildet. Tabelle 4.1 zeigt die aggregierten Endscores für alle untersuchten Web-Architekturen. Die detaillierten Berechnungstabellen der einzelnen Dimensionen sind ebenfalls im Anhang hinterlegt (siehe A.2).

Tabelle 4.1.: Finale Datenmatrix (Scores 1–10) für das TOPSIS-Verfahren

Framework	**Langlebigkeit**	**Wartbarkeit**	**Energie**	**Performance**	**Prod.**	**Team**
Svelte	5,99	8,53	9,59	9,31	7,90	6,00
Vue	7,78	8,15	8,69	7,49	8,16	7,10
React	9,57	8,18	7,75	6,36	8,54	9,87
Angular	7,75	7,15	6,40	5,92	7,30	8,36
VanillaJS	10,00	4,13	9,60	10,00	5,52	4,00

4.3. Analytic Hierarchy Process (AHP) zur Kriteriengewichtung

Der erste Schritt des Entscheidungsmodells besteht in der Bestimmung der Kriteriengewichte. Da die Relevanz von Faktoren wie Energieeffizienz oder Team-Skalierbarkeit je nach Projekttyp variiert, ist eine statische Gewichtung nicht zielführend. Der AHP nach Saaty ermöglicht es, diese subjektiven Präferenzen durch einen systematischen, paarweisen Vergleich in einen mathematisch exakten Prioritätsvektor zu überführen (Mühlbacher & Kaczynski, 2014; Riedl, 2005).

4.3.1. Hierarchische Struktur und paarweiser Vergleich

Das Entscheidungsproblem wird im Modell als einfache Hierarchie abgebildet: Das Gesamtziel (Wahl der optimalen Web-Architektur) steht an der Spitze, gefolgt von den sechs in Kapitel 2.5 definierten Kriterien.

Im entwickelten Artefakt wird der Nutzer durch einen geführten Prozess geleitet, in dem er jeweils zwei Kriterien direkt gegeneinander abwägt (z. B. „Wie wichtig ist Performance im Vergleich zur Langlebigkeit?“). Zur Quantifizierung dieser Urteile wird die fundamentale Skala nach Saaty verwendet, die Werte von 1 (gleiche Wichtigkeit) bis 9 (extreme Wichtigkeit) umfasst (Mühlbacher & Kaczynski, 2014). Die reziproken Werte ($1/3, 1/9$ etc.) werden automatisch für den umgekehrten Vergleich eingesetzt.

4.3.2. Mathematische Ableitung der Gewichte

Die Ergebnisse der Paarvergleiche werden in einer quadratischen $n \times n$ Matrix A zusammengefasst:

$$A = \begin{pmatrix} 1 & a_{12} & \dots & a_{1n} \\ 1/a_{12} & 1 & \dots & a_{2n} \\ \vdots & \vdots & 1 & \vdots \\ 1/a_{1n} & 1/a_{2n} & \dots & 1 \end{pmatrix} \tag{4.2}$$

Zur Berechnung der finalen Gewichte w (Prioritätsvektor) nutzt das Modell die Eigenwertmethode. Dabei wird die Matrix normalisiert, indem jeder Wert durch seine Spaltensumme dividiert wird, woraufhin der arithmetische Mittelwert der Zeilen gebildet wird. Das Ergebnis ist ein Vektor, dessen Summe 1,0 ergibt und der die relative Wichtigkeit jedes Kriteriums darstellt. (Saaty, 2003)

4.3.3. Konsistenzprüfung

Ein wesentlicher Vorteil des AHP gegenüber einfachen Schätzverfahren ist die Möglichkeit, die logische Stimmigkeit der Nutzerangaben zu prüfen. In einer komplexen Matrix könnten widersprüchliche Aussagen auftreten (z. B. A wichtiger als B, B wichtiger als C, aber C wichtiger als A).

Das Modell berechnet hierfür den *Consistency Index* und die daraus resultierende *Consistency Ratio* (CR). Ein CR-Wert von $\leq 0,10$ (10 %) gilt in der Literatur als akzeptabel (Mühlbacher & Kaczynski, 2014; Riedl, 2005). Im Artefakt wird dieser Index

in Echtzeit berechnet, um den Entscheider auf potenzielle Inkonsistenzen in seiner Priorisierung hinzuweisen und somit die Validität der resultierenden Gewichtungsmatrix zu erhöhen.

4.4. Das TOPSIS-Verfahren zur Alternativenbewertung

Nachdem durch den AHP (siehe Abschnitt 4.3) die projektspezifischen Kriteriengewichte ermittelt wurden, erfolgt im zweiten Schritt die Bewertung der technologischen Alternativen mittels TOPSIS. Dieses Verfahren dient dazu, die in der Entscheidungsmatrix (Tabelle 4.1) hinterlegten Daten mathematisch mit den Nutzerpräferenzen zu verknüpfen, um die vierte Forschungsfrage (F4) final zu beantworten.

4.4.1. Prinzip der idealen und anti-idealen Lösung

Das Kernkonzept von TOPSIS basiert auf der Annahme, dass die optimale Architektur diejenige ist, welche die kürzeste geometrische Distanz zur Positive Ideal Solution (PIS) und gleichzeitig die größte Distanz zur Negative Ideal Solution (NIS) aufweist (Madanchian & Taherdoost, 2023). In diesem Modell stellt die PIS eine fiktive Web-Architektur dar, welche in allen sechs Kategorien (z. B. Performance, Energieeffizienz, Team-Skalierbarkeit) den maximal erreichbaren Score von 10,0 erzielt. Die NIS hingegen bildet das theoretische Minimum ab.

4.4.2. Mathematischer Ablauf des Algorithmus

Die Berechnung im entwickelten Artefakt erfolgt in vier sequenziellen Phasen:

1. **Normalisierung der Entscheidungsmatrix:** Da die Scores bereits auf einer einheitlichen Skala von 1 bis 10 vorliegen (siehe Abschnitt 4.2), wird die Matrix normalisiert, um die Vergleichbarkeit zwischen den Kriterien sicherzustellen und die Vektorlänge der Alternativen zu vereinheitlichen.
2. **Gewichtung der Matrix:** Die normalisierten Werte werden mit den aus dem AHP resultierenden Gewichten w_j multipliziert. Dadurch wird sichergestellt, dass Kriterien, die der Nutzer als besonders wichtig eingestuft hat, einen größeren Einfluss auf die finale Distanzberechnung haben (vgl. Madanchian & Taherdoost, 2023).

3. **Bestimmung der Distanzmaße:** Unter Verwendung der euklidischen Distanz berechnet das Modell für jede Web-Architektur (Svelte, React, VanillaJS etc.) zwei Werte:
   - d_i^+: Distanz der Alternative zur positiven Ideallösung.
   - d_i^-: Distanz der Alternative zur negativen Ideallösung.
4. **Berechnung des relativen Nähekoeffizienten (C_i):** Der finale Rangwert ergibt sich aus dem Verhältnis der Distanzen:

$$C_i = \frac{d_i^-}{d_i^+ + d_i^-} \tag{4.3}$$

Der Koeffizient C_i liegt immer zwischen 0 und 1. Eine Architektur mit einem Wert nahe 1 kommt der idealen Lösung am nächsten und wird vom Modell als bevorzugte Wahl empfohlen. Durch diesen mathematischen Prozess wird die oft subjektiv geprägte Diskussion um Web-Architekturen durch ein transparentes, auf Distanzmessungen basierendes Ranking ersetzt (Madanchian & Taherdoost, 2023).

5. Implementierung des Artefakts

In diesem Kapitel wird die technische Realisierung des Entscheidungsmodells beschrieben. Das Artefakt wurde als webbasierte Applikation umgesetzt, die den theoretischen AHP-TOPSIS-Ansatz in ein interaktives Werkzeug für Software-Architekten überführt. Bei der technischen Implementierung des Artefakts wurde das Large Language Model *Gemini* (Google, 2026) als unterstützendes Entwicklungswerkzeug herangezogen. Die Nutzung umfasste primär die syntaktische Ausformulierung von Code-Blöcken, das Debugging sowie die Generierung von Boilerplate-Code für das Node.js-Backend. Die methodische Konzeption, das Architekturdesign, die Entwicklung der mathematischen AHP-TOPSIS-Logik sowie die finale Überprüfung und Integration des Quellcodes erfolgten vollständig eigenverantwortlich. Der vollständige Quellcode ist im GitHub-Repository unter https://github.com/stfnklc/Entscheidungsmodell_Webframeworks.git hinterlegt.

5.1. Systemarchitektur und Technologiestack

Bei der Entwicklung des Artefakts wurde ein bewusster Fokus auf eine schlanke und effiziente Systemarchitektur gelegt. Um zusätzliche Abhängigkeiten zu minimieren und eine performante Applikation zu gewährleisten, erfolgte die technische Umsetzung primär auf Basis nativer Web-Technologien.

- **Backend:** Als Serverumgebung dient *Node.js* mit dem Framework *Fastify*. Das Backend fungiert primär als statischer Dateiserver und stellt die Anwendung bereit.
- **Frontend:** Die Benutzeroberfläche wurde vollständig mit nativen Web-Technologien (HTML5, CSS3 und Vanilla JS) realisiert. Durch den Verzicht auf schwere Frontend-Frameworks wird eine minimale Ladezeit des Werkzeugs gewährleistet.
- **Modulare Struktur:** Die mathematische Logik wurde strikt von der UI-Interaktion getrennt.

5.2. Workflow und Benutzeroberfläche

5.2.1. Landing Page

Bevor der mathematische Prozess beginnt, wird der Nutzer auf einer zentralen Landing Page empfangen. Diese dient dazu, den Kontext der Anwendung zu erläutern und eine Brücke zwischen der theoretischen Problemstellung und der praktischen Lösung zu schlagen.

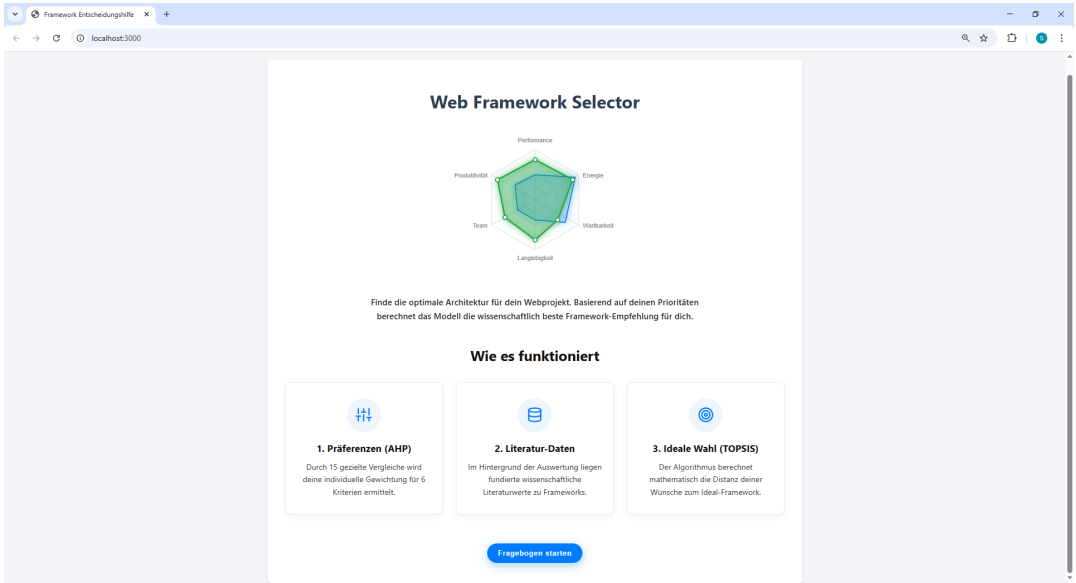

Abbildung 5.1.: Landing Page des Artefakts mit Erläuterung der Zielsetzung

Wie in Abbildung 5.1 zu sehen, wird dem Anwender kurz dargelegt, dass die Wahl einer Web-Architektur (Native vs. Framework) kein binäres Problem ist, sondern von spezifischen Projektanforderungen abhängt. Dem Nutzer wird erklärt, dass das Werkzeug zwei methodische Schritte kombiniert:

1. Die Erfassung seiner persönlichen Prioritäten (subjektive Sicht).
2. Den Abgleich dieser Prioritäten mit wissenschaftlich fundierten Daten (objektive Sicht).

5.2.2. Phase 1: Subjektive Priorisierung mittels AHP

Der Einstieg in die Anwendung erfolgt über den paarweisen Vergleich der sechs Hauptkriterien. Wie in Abbildung 5.2 dargestellt, wird jedes Kriterium direkt gegen ein anderes abgewogen.

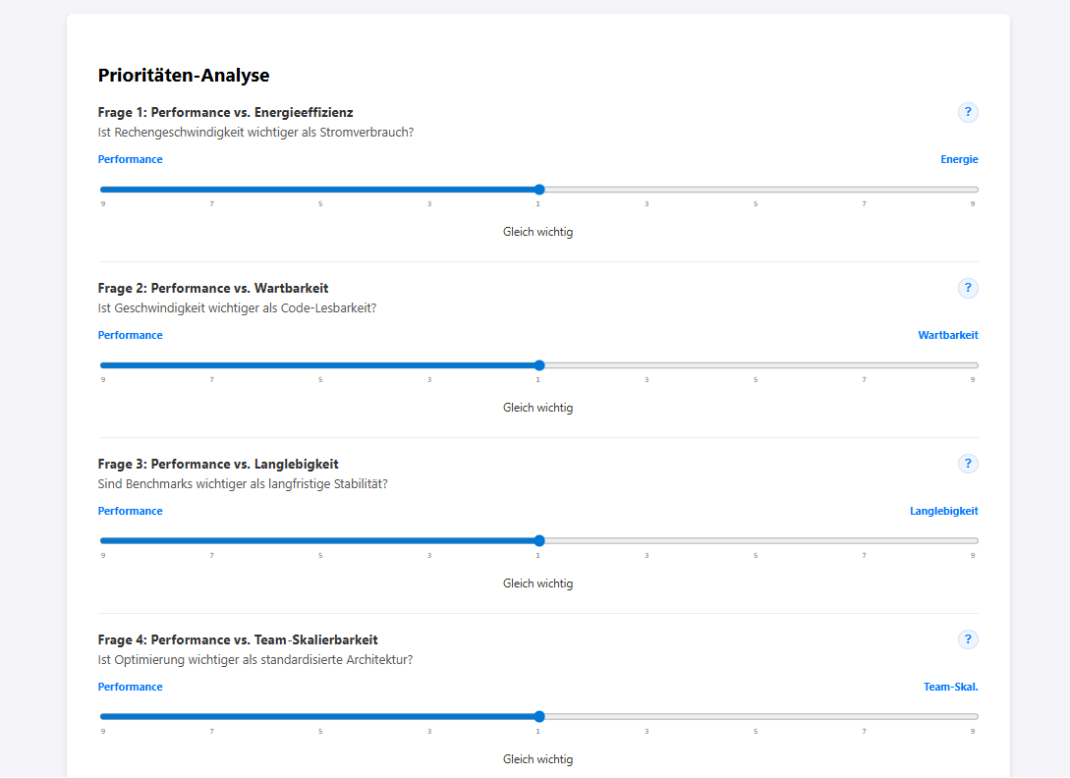

Abbildung 5.2.: Benutzeroberfläche für den paarweisen Kriterienvergleich (AHP)

Die Umsetzung nutzt Schieberegler, die intern auf die Saaty-Skala (1 bis 9) abgebildet werden. Der Nutzer bestimmt hierbei die relative Wichtigkeit (z. B. Performance gegenüber Langlebigkeit). Die Logik für den AHP aktualisiert dabei simultan die reziproke Matrix und berechnet den Konsistenzindex, um logische Widersprüche in der Eingabe abzufangen.

5.2.3. Visualisierung der Kriteriengewichte

Nach Abschluss der Paarvergleiche berechnet das Modell den Prioritätsvektor. Um dem Nutzer Transparenz über seine eigene Gewichtung zu geben, werden die berechneten Werte visuell aufbereitet (siehe Abbildung 5.3).

Abbildung 5.3.: Visualisierung der berechneten Kriteriengewichte

Dieser Schritt dient der Validierung: Der Entscheider kann prüfen, ob die mathematisch abgeleiteten Gewichte seinem tatsächlichen Projektprofil entsprechen.

5.2.4. TOPSIS-Ranking der Frameworks

Im finalen Schritt werden die individuellen Gewichte mit den gewichteten Endscores aus der Literaturanalyse (siehe Abschnitt 4.2.2) verrechnet. Das Ergebnis wird als Rangliste präsentiert (Abbildung 5.4).

Der implementierte TOPSIS-Algorithmus ermittelt für jede Technologie (Svelte, Vue, React, Angular, VanillaJS) den relativen Nähekoeffizienten zur idealen Lösung. Die Architektur mit dem höchsten Wert wird als mathematisch optimal für die getroffenen Annahmen hervorgehoben.

Um die mathematischen Ergebnisse der TOPSIS-Berechnung für den Nutzer greifbar und interpretierbar zu machen, verfügt die Anwendung über eine integrierte Datenvisualisierung. Wie in Abbildung 5.5 dargestellt, wird das empfohlene Sieger-Framework in einem Spinnennetz- bzw. Netzdiagramm (implementiert über die Bibliothek *Chart.js*)

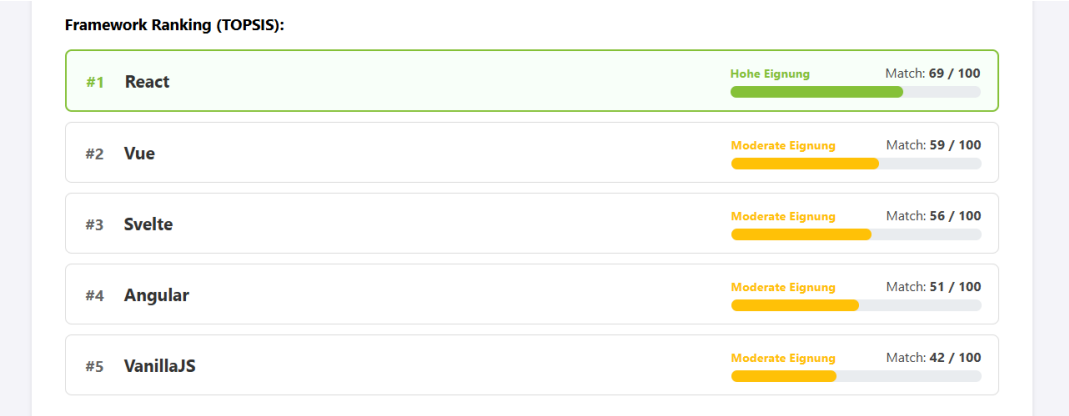

Abbildung 5.4.: Ergebnis: Ranking der Frameworks

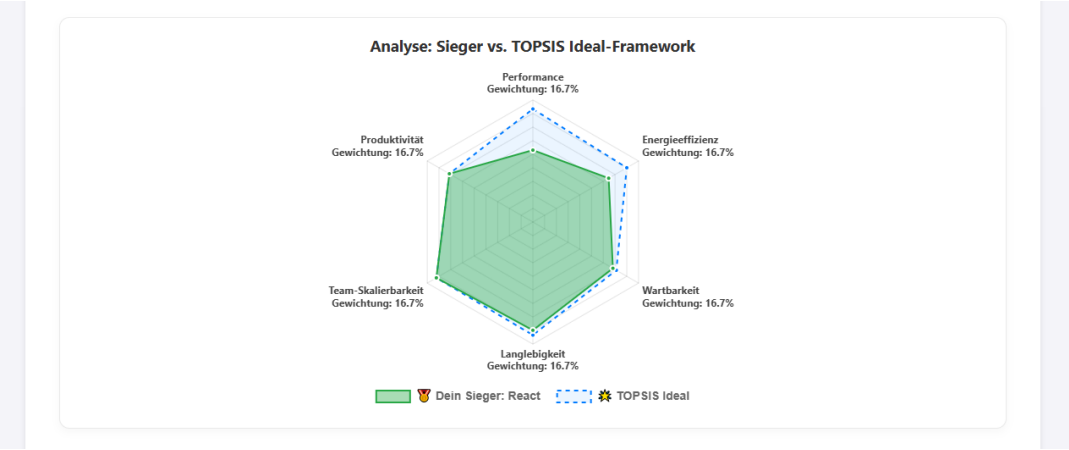

Abbildung 5.5.: Vergleich vom besten Framework mit TOPSIS-Ideal

der theoretischen Ideallösung gegenübergestellt. Diese grafische Aufbereitung veranschaulicht nicht nur den aggregierten Nähekoeffizienten, sondern ermöglicht es dem Anwender, auf einen Blick zu erfassen, in welchen spezifischen Kriterien-Dimensionen (wie z. B. Performance oder Langlebigkeit) die gewählte Architektur dem absoluten Ideal nahekommt und wo gegebenenfalls noch Kompromisse eingegangen werden müssen.

6. Evaluierung und Diskussion

Nach der technischen Implementierung des Artefakts erfolgt in diesem Kapitel die Validierung des Entscheidungsmodells. Ziel ist es, die Sensitivität und Praxistauglichkeit des hybriden AHP-TOPSIS-Ansatzes anhand von kontrastierenden Anwendungsszenarien zu prüfen. Dabei wird untersucht, wie sich unterschiedliche Priorisierungen auf das finale Ranking der Web-Architekturen auswirken.

6.1. Szenariobasierte Evaluierung

Um die vierte Forschungsfrage (F4) zu beantworten, wurden praxisnahe Szenarien definiert, die typische Entscheidungskonflikte in der Software-Architektur widerspiegeln. Die Eingabewerte für den AHP basieren auf der Saaty-Skala (1 bis 9), wobei positive Werte eine Präferenz für das zweitgenannte Kriterium und negative Werte eine Präferenz für das erstgenannte Kriterium ausdrücken.

6.1.1. Szenario 1: Enterprise Core-System (Online-Banking)

Dieses Szenario simuliert die Neuentwicklung eines Kundenportals für ein Kreditinstitut. Die Priorität liegt auf einer stabilen, langfristig verfügbaren Architektur, die von großen, fluktuierenden Entwicklerteams gewartet werden kann. Kurzfristige Produktivität ist aufgrund der strategischen Bedeutung des Projekts nachrangig.

- **Anforderungsprofil:** Höchster Fokus auf *Langlebigkeit* (Platz 1), gefolgt von *Wartbarkeit* (Platz 2) und *Team-Skalierbarkeit* (Platz 3).
- **AHP-Eingaben (Auszug):** Starke Präferenz für Langlebigkeit gegenüber Performance (+6) und Produktivität (-6). Moderate Präferenz für Wartbarkeit gegenüber Produktivität (-4).

Ergebnis und Analyse:

In diesem Szenario identifiziert das Modell **React** als die optimale Wahl (siehe Abbildung 6.1), gefolgt von Angular und Vue. Dieses Ergebnis ist mathematisch auf die hohen Scores von React in den Kategorien Team-Skalierbarkeit (9,87) und Langlebigkeit

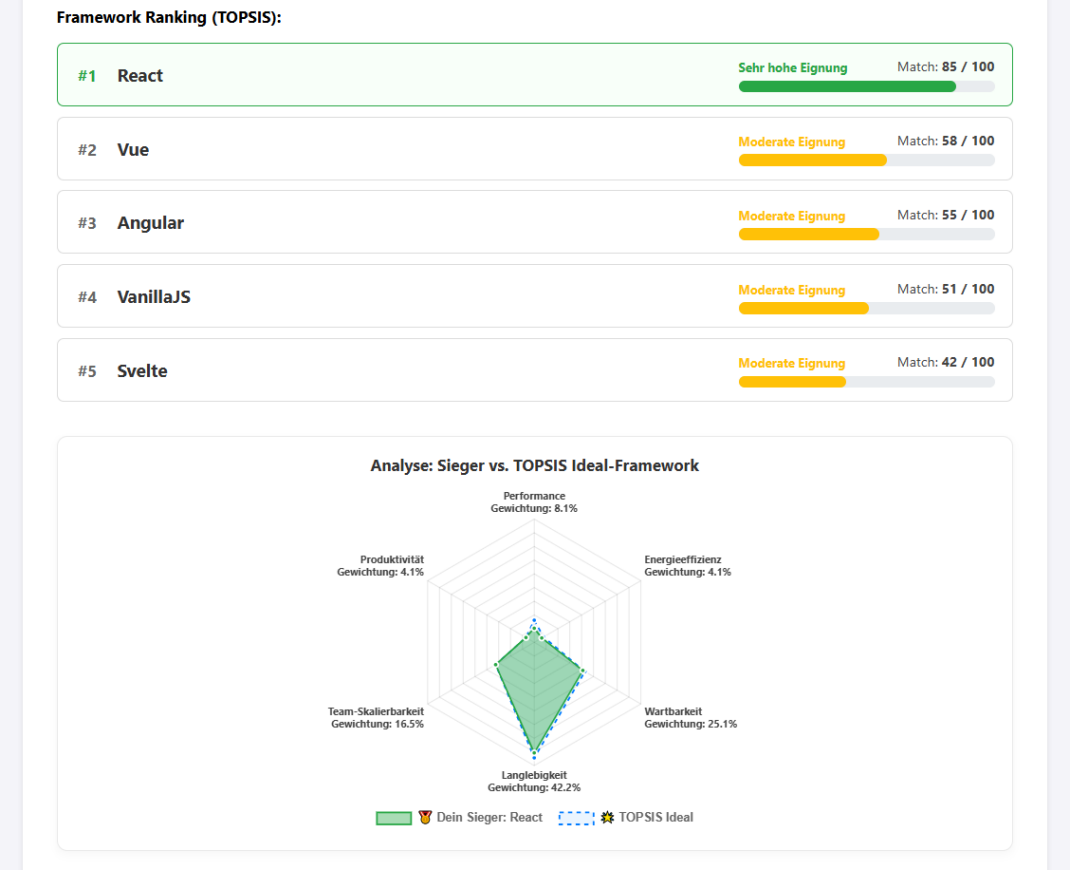

Abbildung 6.1.: Ergebnisauswertung: React als Sieger im Enterprise-Szenario

(9,57) zurückzuführen. Obwohl VanillaJS in der technischen Performance dominiert, fällt es aufgrund der unzureichenden Wartbarkeit (4,13) und Team-Skalierbarkeit (4,00) im Ranking weit zurück. Das Modell spiegelt hierbei die ökonomische Realität wider, in der das Risiko eines technologischen Ausfalls oder eines Fachkräftemangels schwerer wiegt als technischer Overhead (vgl. Duvander & Romhagen, 2019; Repolust, 2025).

6.1.2. Szenario 2: Startup MVP (Minimum Viable Product)

Das zweite Szenario beschreibt ein Startup, das unter hohem Zeit- und Budgetdruck einen Prototypen entwickeln muss. Ziel ist es, schnellstmöglich Features zu liefern

(Time-to-Market) und gleichzeitig eine hohe User Experience durch gute Performance zu gewährleisten.

- **Anforderungsprofil:** Höchster Fokus auf *Produktivität* (Platz 1), gefolgt von *Performance* (Platz 2) und *Wartbarkeit* (Platz 3).
- **AHP-Eingaben (Auszug):** Sehr starke Präferenz für Produktivität gegenüber Team-Skalierbarkeit (+6) und Langlebigkeit (+6). Leichte Präferenz für Produktivität gegenüber Performance (+2).

Ergebnis und Analyse:

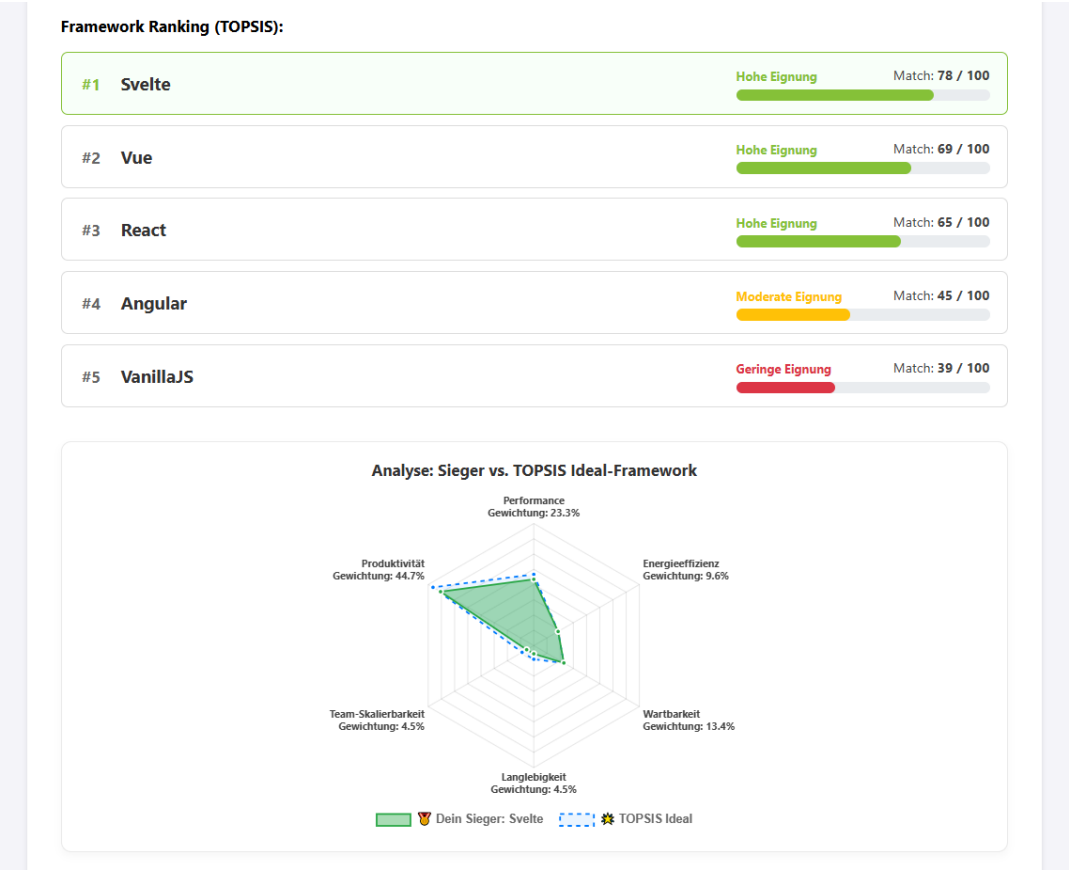

Abbildung 6.2.: Ergebnisauswertung: Svelte als Sieger im MVP-Szenario

Das Modell empfiehlt in diesem Szenario **Svelte** als bevorzugte Lösung (siehe Abbildung 6.2). Svelte profitiert hier von seiner einzigartigen Positionierung: Es bietet

eine hohe Produktivität durch ein modernes Komponentenmodell (7,90), ohne die Performance-Einbußen klassischer Frameworks (Performance-Score: 9,31). React bleibt aufgrund der etwas geringeren Performance-Werte (6,36) auf dem zweiten Platz, während VanillaJS trotz der Maximalwerte in Performance und Energieeffizienz aufgrund der niedrigen Produktivitätsbewertung (5,52) für ein schnelles Minimum Viable Product (MVP) als weniger geeignet eingestuft wird.

6.1.3. Sensitivitätsanalyse und Modellrobustheit

Um die Robustheit des Modells zu prüfen, wurde ergänzend eine punktuelle Sensitivitätsanalyse durchgeführt. Es wurde untersucht, wie stark sich das Ranking ändert, wenn die Gewichtungen leicht variiert werden.

Wird im MVP-Szenario (Szenario 2) die Wichtigkeit der Performance schrittweise auf der Saaty-Skala erhöht und die Produktivität leicht abgesenkt, zeigt das Modell ein stabiles Verhalten: Svelte behält zunächst seine Spitzenposition, während VanillaJS den Abstand zu React verringert und auf den zweiten Platz vorrückt. Erst bei einer extremen, einseitigen Gewichtung der reinen Performance ($> 60\,\%$ der relativen Gesamtgewichtung) verdrängt VanillaJS Svelte von Platz 1. Diese Analyse demonstriert, dass das TOPSIS-Ranking gegenüber leichten Schwankungen und Unschärfen in den subjektiven Eingaben robust ist und nicht zu erratischen Sprüngen neigt.

6.2. Diskussion der Modellergebnisse

Die Evaluierung verdeutlicht, dass das Modell erfolgreich zwischen technischen Idealen und organisatorischen Notwendigkeiten differenziert.

Ein zentraler Diskussionspunkt ist die Rolle von *Svelte* als „Brückentechnologie“. In beiden Szenarien zeigt sich, dass Svelte technische Effizienz (nahe VanillaJS) mit moderner Entwicklererfahrung (nahe React) kombiniert. Dass Svelte im Enterprise-Szenario dennoch hinter React zurückbleibt, liegt an der mathematischen Berücksichtigung der *Team-Skalierbarkeit*. Dies validiert die theoretische Annahme aus Kapitel 3.2, dass ein technologisch überlegenes System organisatorisch riskant sein kann, wenn das Ökosystem (Jobmarkt, Community) noch nicht gesättigt ist (Bauer, 2021; Pano et al., 2018).

Des Weiteren bestätigt die Evaluierung den „Framework-Trade-off“: Die Entscheidung für eine Abstraktionsschicht ist primär eine Entscheidung für Wartbarkeit und gegen maximale Ressourceneffizienz. Das Artefakt macht diesen oft intuitiv getroffenen Tradeoff durch die TOPSIS-Distanzmessung erstmals quantifizierbar und transparent.

6.3. Limitationen des Modells

Trotz der konsistenten Ergebnisse müssen methodische Limitationen berücksichtigt werden:

1. **Datenaktualität:** Die Qualität des Rankings steht und fällt mit der Objektivität der hinterlegten Benchmark-Scores. Da sich Frameworks stetig weiterentwickeln (z. B. durch neue Rendering-Konzepte in React oder Angular), muss die Datenbasis (Kapitel 4.2.2) als temporäre Momentaufnahme betrachtet werden.
2. **Subjektivität im AHP:** Das Modell setzt voraus, dass der Anwender die Kriterienbedeutungen korrekt interpretiert, um aussagekräftige Paarvergleiche im AHP durchzuführen.
3. **Umfang und Validierung der Szenarien:** Die Evaluierung beschränkt sich auf zwei Szenarien, die zudem synthetisch konstruiert wurden. Eine externe Validierung durch Fallstudien in der Industrie oder eine systematische Überprüfung der Empfehlungen durch externe Software-Architekten hat im Rahmen dieser Arbeit nicht stattgefunden. Für den Umfang einer Bachelorarbeit dient dies als ausreichender *Proof of Concept*, jedoch wären groß angelegte empirische Feldtests notwendig, um die Modellempfehlungen in der wirtschaftlichen Praxis final zu validieren.

7. Fazit und Ausblick

Die vorliegende Arbeit verfolgte das Ziel, ein mathematisch fundiertes Entscheidungsmodell zu entwickeln, das Entwickler und Software-Architekten bei der Wahl zwischen nativen Web-Technologien und Framework-basierten Architekturen unterstützt. Durch die Kombination einer systematischen Literaturanalyse mit dem hybriden AHP-TOPSIS-Verfahren konnte ein Artefakt geschaffen werden, das sowohl technische Messwerte als auch organisatorische Rahmenbedingungen berücksichtigt.

7.1. Zusammenfassung der Ergebnisse

Die Beantwortung der untergeordneten Forschungsfragen führt zu einem differenzierten Gesamtbild der aktuellen Web-Entwicklungslandschaft:

- **Forschungsfrage F1 (Kriterien):** Es konnten sechs zentrale Dimensionen identifiziert werden, die für die Architekturauswahl entscheidend sind. Während Performance und Energieeffizienz die technische Effizienz abbilden, adressieren Wartbarkeit, Langlebigkeit, Team-Skalierbarkeit und Produktivität die softwareökonomischen und organisatorischen Anforderungen.
- **Forschungsfrage F2 (Framework-Einfluss):** Die Analyse zeigt einen klaren Trade-off. Frameworks erhöhen durch Abstraktionsschichten wie den Virtual DOM zwangsläufig den Ressourcenverbrauch und die Latenz. Gleichzeitig bieten sie jedoch durch Standardisierung und fertige Komponenten eine signifikant höhere Produktivität in komplexen Umgebungen.
- **Forschungsfrage F3 (Stabilität & Abhängigkeiten):** Native Technologien weisen aufgrund der zugrunde liegenden Web-Standards eine überlegene langfristige Stabilität auf. Frameworks hingegen bergen Risiken durch technologische Obsoleszenz und Breaking Changes bei Major-Updates, bieten jedoch durch ihre großen Communities eine höhere Sicherheit bei der Personalgewinnung.
- **Forschungsfrage F4 (Modellbildung):** Das entwickelte hybride Modell aus AHP und TOPSIS erweist sich als geeignet, um die identifizierten Kriterien in eine gewichtete Entscheidungsmatrix zu überführen. Es ermöglicht eine mathematisch herleitbare Empfehlung, die über intuitive Bauchentscheidungen hinausgeht.

7.2. Kritische Würdigung

Das entwickelte Artefakt stellt ein wertvolles Hilfsmittel dar, um die oft subjektiv geführte Debatte über „das beste Framework“ zu objektivieren. Besonders die Erkenntnis, dass Technologien wie Svelte einen leistungsfähigen Mittelweg zwischen nativer Effizienz und Framework-Komfort bieten, unterstreicht die Relevanz einer kriterienbasierten Analyse.

Dennoch bleibt festzuhalten, dass ein solches Modell die menschliche Expertise nicht vollständig ersetzen kann. Die Qualität der Ergebnisse ist maßgeblich von der präzisen Einschätzung der Projektanforderungen durch den Nutzer abhängig. Zudem muss die zugrunde liegende Benchmark-Datenbasis kontinuierlich gepflegt werden, um technologische Sprünge (z. B. Optimierungen in der Browser-Engine oder neue Rendering-Paradigmen) zeitnah abzubilden.

7.3. Ausblick

Die Zukunft der Web-Architekturen deutet auf eine weitere Annäherung zwischen nativen Standards und Framework-Konzepten hin. Technologien wie *Web Components* gewinnen an Reife und könnten langfristig dazu führen, dass Frameworks weniger als monolithische Laufzeitumgebungen, sondern eher als spezialisierte Build-Tools fungieren (wie bereits im Ansatz von Svelte sichtbar).

Zukünftige Erweiterungen des Modells könnten automatisierte Daten-Schnittstellen zu Plattformen wie GitHub oder npm einbeziehen, um die Scores für Langlebigkeit und Team-Skalierbarkeit in Echtzeit zu aktualisieren. Darüber hinaus wäre die Integration von Künstlicher Intelligenz denkbar, die auf Basis von Projektbeschreibungen, mittels LLM-basierter Anforderungsanalyse automatisch Vorschläge für die AHP-Gewichtung generiert.

Abschließend lässt sich konstatieren, dass es in der modernen Web-Entwicklung keine „Einheitslösung“ gibt. Der bewusste Verzicht auf Framework-Abstraktionen kann in Zeiten von *Green IT* und Performance-Fokus ebenso eine valide strategische Entscheidung sein wie die Wahl eines etablierten Frameworks für skalierbare Enterprise-Systeme.

Literatur

Bauer, M. (2021). *Hello Framework! A heuristic method for choosing front-end JavaScript frameworks* [Bachelor Thesis]. Uppsala University, Department of Information Technology.

Borze, A., Poszet, O., Cuc, A.-M., & Vari-Kakas, S. (2025). Comparative Study Between React and Svelte: Performance and Usability. *2025 18th International Conference on Engineering of Modern Electric Systems (EMES)*, 1–4. https://doi.org/10.1109/EMES65692.2025.11045600

Devographics. (2025). State of JavaScript [Zugriff: 11. Feb 2026]. https://2025.stateofjs.com/en-US

Diniz-Junior, R. N., Figueiredo, C. C. L., De S.Russo, G., Bahiense-Junior, M. R. G., Arbex, M. V., Dos Santos, L. M., Da Rocha, R. F., Bezerra, R. R., & Giuntini, F. T. (2022). Evaluating the performance of web rendering technologies based on JavaScript: Angular, React, and Vue. *2022 XVLIII Latin American Computer Conference (CLEI)*, 1–9. https://doi.org/10.1109/CLEI56649.2022.9959901

Duvander, J., & Romhagen, O. (2019). *What affects the choice of a JavaScript framework: Interviews with developers* [Bachelor Thesis]. Jönköping University, School of Engineering.

Emmanni, P. S. (2023). Comparative Analysis of Angular, React, and Vue.js in Single Page Application Development. *International Journal of Science and Research (IJSR)*, *12*(6), 2971–2974. https://doi.org/https://dx.doi.org/10.21275/SR24401230015

Finckelsen, C. (2024). *Environmental Impact Between Client-Side Rendering And Server-Side Rendering In Popular Front-end Frameworks: A comparative analysis* [Master of Science Programme in Interaction Technology and Design]. Umeå University.

Google. (2026). *Gemini* [Zugriff: 18. Februar 2026]. https://gemini.google.com/app

Krause, S. (2025). js-framework-benchmark [Zugriff: 9. Feb 2026]. https://github.com/krausest/js-framework-benchmark

Kučak, D., Blažević, M., & Bele, D. (2025). Comparative Analysis of Scripting Frameworks for Web Application Development. *Computer Science On-line Conference (CSOC 2025)*, 149–161. https://doi.org/10.1007/978-3-032-00236-5_12

Levlin, M. (2020). *DOM benchmark comparison of the front-end JavaScript frameworks React, Angular, Vue, and Svelte* [Master's Thesis in Computer Science]. Åbo Akademi University.

Madanchian, M., & Taherdoost, H. (2023). A comprehensive guide to the TOPSIS method for multi-criteria decision making. *Sustainable Social Development*, *1*(1). https://doi.org/10.54517/ssd.v1i1.2220

Mohammad, S., Jobair, A. A., & Abedeen, I. (2024). Decoding Code Quality: A Software Metric Analysis of Open-Source JavaScript Projects. *Proceedings of the 19th International Conference on Evaluation of Novel Approaches to Software Engineering - Volume 1: ENASE*, 63–74. https://doi.org/10.5220/0012618800003687

Mozilla Developer Network. (2025). *Document Object Model (DOM)* [Zugriff: 3. Februar 2026]. https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model

Mühlbacher, A. C., & Kaczynski, A. (2014). Der Analytic Hierarchy Process (AHP): Eine Methode zur Entscheidungsunterstützung im Gesundheitswesen. *PharmacoEconomics - German Research Articles*, *11*, 119–132. https://doi.org/10.1007/s40275-014-0011-8

npm, Inc. (2025). npm Registry: Download Statistics for React, Vue, Angular, Svelte [Zugriff: 12. Februar 2026]. https://www.npmjs.com/

Ollila, R., Mäkitalo, N., & Mikkonen, T. (2022). Modern Web Frameworks: A Comparison of Rendering Performance. *Journal of Web Engineering*, *21*(3), 789–813. https://doi.org/10.13052/jwe1540-9589.21311

Ouchaib, J. (2024). *Benchmarking Modern Frontend Frameworks: A Comparative Performance Analysis* [Master's Degree Thesis]. Politecnico di Torino.

Pano, A., Graziotin, D., & Abrahamsson, P. (2018). Factors and actors leading to the adoption of a JavaScript framework. *Empirical Software Engineering*, *23*, 3503–3534. https://doi.org/10.1007/s10664-018-9613-x

Persson, M. (2020). *JavaScript DOM Manipulation Performance: Comparing Vanilla JavaScript and Leading JavaScript Front-end Frameworks* [Bachelor's degree in software engineering]. Blekinge Institute of Technology.

Putra, F. P. E., Hasbullah, Muslim, F., & Paradina, R. (2025). Technical Performance Comparison of Modern Frontend Frameworks: Study on Svelte, React, and Vue. *Brilliance: Research of Artificial Intelligence*, *5*(1). https://doi.org/10.47709/brilliance.v5i1.6133

Rathinam, S. (2022). Analysis and Comparison of Different Frontend Frameworks. *Applications and Techniques in Information Security*, 243–257. https://doi.org/10.1007/978-981-99-2264-2_19

Repolust, L. (2025). *Frontend-Technologien und deren Weiterentwicklung: Vergleich von Angular, React und Vue.js zur Feststellung der Eignung anhand von Kriterien der Abwärtskompatibilität für Legacy-Projekte* [Masterarbeit]. Fachhochschule CAMPUS 02 Graz.

Riedl, R. (2005). Der Analytic Hierarchy Process: Ein geeignetes Verfahren für komplexe Entscheidungen in der Wirtschaftsinformatik? *HMD Praxis der Wirtschaftsinformatik*, *246*(2), 104–113.

Saaty, T. L. (2003). Decision-making with the AHP: Why is the principal eigenvector necessary. *European Journal of Operational Research*, *145*(1), 85–91. https://doi.org/https://doi.org/10.1016/S0377-2217(02)00227-8

Sinha, K., & Jana, D. S. (2024). The Role of JavaScript Frameworks in Performance Optimization: A Comparative Study. *MAT Journals: Journal of Software Engineering and Simulation*, *10*(3), 16–23.

StackOverflow. (2025). 2025 Stack Overflow Developer Survey [Zugriff: 11. Feb 2026]. https://survey.stackoverflow.co/

W3C. (1998). *What is the Document Object Model?* [Zugriff: 3. Februar 2026]. https://www.w3.org/TR/WD-DOM/introduction.html

Wadholm, M. (2023). *Green and Sustainable JavaScript: A study into the impact of framework usage* [Bachelor of Science in Software Engineering]. Blekinge Institute of Technology.

Xu, W. (2021). *Benchmark Comparison of JavaScript Frameworks: React, Vue, Angular and Svelte* [Master of Science Interactive Digital Media]. University of Dublin.

Zozas, I., Bibi, S., Ampatzoglou, A., & Sarigiannidis, P. (2019). Estimating the maintenance effort of JavaScript Applications. *2019 45th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*, 212–219. https://doi.org/10.1109/SEAA.2019.00042

A. Quellengewichtung und Datengrundlage

A.1. Detaillierte Quellengewichtung

Die Gewichtung der Quellen basiert auf dem arithmetischen Mittel der Kriterien Aktualität, Scope und wissenschaftlicher Grad (Skala 0,1 bis 1,0).

Tabelle A.1.: Datenerhebung: Quellen-Gewichtung

Quelle	**Aktualität**	**Scope**	**Wiss. Grad**	**Gewicht (Ø)**
Putra et al. (2025)	1,00	0,80	1,00	0,93
Borze et al. (2025)	1,00	0,80	1,00	0,93
Kučak et al. (2025)	1,00	0,80	1,00	0,93
Stack Overflow (2025)	1,00	1,00	0,60	0,87
State of JS (2025)	1,00	1,00	0,60	0,87
GitHub/Npm (2025)	1,00	1,00	0,60	0,87
Repolust (2025)	1,00	0,60	0,80	0,80
Krause (2025)	1,00	1,00	0,60	0,87
Sinha & Jana (2024)	0,90	0,80	1,00	0,90
Ouchaib (2024)	0,90	0,60	0,80	0,77
Diniz-Junior et al. (2022)	0,70	0,80	1,00	0,83
Bauer (2021)	0,60	0,60	0,70	0,63
Xu (2021)	0,60	0,60	0,80	0,67
Levlin (2020)	0,50	0,60	0,70	0,60
Persson (2020)	0,50	0,60	0,70	0,60
Duvander (2019)	0,40	0,60	0,70	0,57
Pano & Graziotin (2018)	0,30	0,80	1,00	0,70
Finckelsen (2024)	0,90	0,80	0,80	0,83
Ollila et al. (2022)	0,70	1,00	1,00	0,90
Wadholm (2023)	0,80	0,80	0,70	0,77

A.2. Einzelscores der Kriterien-Dimensionen

Die folgenden Tabellen zeigen die Extraktion der Scores (1–10) aus den Quellen und die Berechnung der gewichteten Endscores für die finale TOPSIS-Matrix.

Tabelle A.2.: Datenerhebung: Team-Skalierbarkeit

Quelle	**Gewicht**	**Svelte**	**Vue**	**React**	**Angular**	**Vanilla**
Pano (2018)	0,70	-	6,5	9,5	9,0	-
Repolust (2025)	0,80	-	7,0	10,0	8,0	-
Bauer (2021)	0,63	6,0	8,0	10,0	8,0	-
Duvander (2019)	0,57	-	7,0	10,0	8,5	-
Endscore		**6,00**	**7,10**	**9,87**	**8,36**	**4,00**

Tabelle A.3.: Datenerhebung: Langlebigkeit

Quelle	**Gewicht**	**Svelte**	**Vue**	**React**	**Angular**	**Vanilla**
Stack Overflow (2025)	0,87	6,5	7,5	10,0	8,0	10,0
State of JS (2025)	0,87	6,5	7,5	10,0	8,0	10,0
Repolus (2025)	0,80	-	6,0	7,5	5,5	-
Bauer (2021)	0,63	6,5	8,0	10,0	8,5	-
Pano (2018)	0,70	5,5	9,0	10,0	8,5	-
GitHub/Npm (2025)	0,87	5,0	9,0	10,0	7,0	-
Duvander (2019)	0,57	-	7,5	9,5	9,5	10,0
Endscore		**5,99**	**7,78**	**9,57**	**7,75**	**10,00**

Tabelle A.4.: Datenerhebung: Performance

Quelle	**Gewicht**	**Svelte**	**Vue**	**React**	**Angular**	**Vanilla**
Putra et al. (2025)	0,93	10,0	8,0	7,0	-	-
Krause (2025)	0,80	9,0	8,5	7,5	6,5	10,0
Sinha (2024)	0,90	9,0	8,0	7,0	6,0	-
Xu (2021)	0,67	9,0	8,0	7,0	6,0	-
Ouchaib (2024)	0,77	10,0	9,0	6,0	7,0	-
Diniz-Junior et al. (2022)	0,83	-	9,0	5,0	4,0	-
Kučak et al. (2025)	0,93	9,0	7,0	6,0	-	-
Ollila et al. (2022)	0,90	10,0	5,0	4,0	6,0	-
Levlin (2020)	0,60	8,0	6,0	7,0	5,0	-
Persson (2020)	0,60	-	6,0	8,0	7,0	10,0
Endscore		**9,31**	**7,49**	**6,36**	**5,92**	**10,00**

Tabelle A.5.: Datenerhebung: Energieeffizienz

Quelle	**Gewicht**	**Svelte**	**Vue**	**React**	**Angular**	**Vanilla**
Putra et al. (2025)	0,93	10,0	8,0	6,0	-	-
Diniz-Junior et al. (2022)	0,83	-	9,0	8,0	7,0	9,0
Persson, (2020)	0,60	-	8,0	8,0	7,0	10,0
Xu (2021)	0,67	8,0	9,0	10,0	6,0	-
Sinha (2024)	0,90	10,0	9,0	8,0	6,0	-
Finckelsen (2024)	0,83	-	10,0	8,0	6,0	-
Krause (2025)	0,80	9,5	8,5	7,5	6,5	10,0
Wadholm (2023)	0,77	-	-	8,0	-	9,5
Kučak et al. (2025)	0,93	10,0	8,0	7,0	-	-
Endscore		**9,59**	**8,69**	**7,75**	**6,40**	**9,60**

Tabelle A.6.: Datenerhebung: Wartbarkeit

Quelle	**Gewicht**	**Svelte**	**Vue**	**React**	**Angular**	**Vanilla**
Borze (2025)	0,93	7,0	-	9,0	-	-
Stack Overflow (2025)	0,87	9,0	8,5	8,0	7,0	5,0
State of JS (2025)	0,87	9,0	8,5	8,0	7,0	-
Repolust (2025)	0,80	-	6,5	7,5	5,5	4,0
Bauer (2021)	0,63	9,5	8,5	8,5	8,0	-
Duvander (2019)	0,57	-	9,0	8,0	9,0	3,0
Endscore		**8,53**	**8,15**	**8,18**	**7,15**	**4,13**

Tabelle A.7.: Datenerhebung: Produktivität

Quelle	**Gewicht**	**Svelte**	**Vue**	**React**	**Angular**	**Vanilla**
Borze (2025)	0,93	8,0	-	9,0	-	-
Pano (2018)	0,70	-	8,5	8,0	7,0	-
Stack Overflow (2025)	0,87	8,0	8,5	8,5	8,0	6,0
State of JS (2025)	0,87	8,0	8,5	8,5	7,5	-
Repolust (2025)	0,80	-	7,0	9,0	7,0	5,0
Bauer (2021)	0,63	7,5	8,0	8,5	7,5	-
Duvander (2019)	0,57	-	8,5	8,0	6,5	-
Endscore		**7,90**	**8,16**	**8,54**	**7,30**	**5,52**

Erklärung

Ich versichere, die von mir vorgelegte Arbeit selbstständig verfasst zu haben. Alle Stellen, die wörtlich oder sinngemäß aus veröffentlichten oder nicht veröffentlichten Arbeiten anderer oder der Verfasserin/des Verfassers selbst entnommen sind, habe ich als entnommen kenntlich gemacht. Sämtliche Quellen und Hilfsmittel, die ich für die Arbeit benutzt habe, sind angegeben. Die Arbeit hat mit gleichem Inhalt bzw. in wesentlichen Teilen noch keiner anderen Prüfungsbehörde vorgelegen.

19.02.26, Alwersbach

Ort, Datum

S. Nikolic

Unterschrift